

Aufgabe 1: Metriken

1. Konfigurationen

Wir haben neben der Basisversion vier weitere Varianten des Programms `primesum.c` erstellt und getestet. Die Varianten unterscheiden sich durch den verwendeten Datentyp (`int64_t` vs. `__int128`), die Modulo-Berechnung und die Compiler-Optimierung.

- **v1_base**: `int64_t`, Modulo $10^9 + 7$, **-O0** (Basisversion).
- **v2_opt**: `int64_t`, Modulo $10^9 + 7$, **-O3**.
- **v3_fastmod**: `int64_t`, Modulo 16^8 (bzw. $0x100000000$), **-O3**.
- **v4_int128**: `__int128`, Modulo $10^9 + 7$, **-O3**.
- **v5_no_opt**: `int64_t`, Modulo 16^8 , **-O0**.

2. Hypothesen (Erwartungen)

Vor der Messung hatten wir folgende Erwartungen:

1. **Optimierung**: Wir erwarten, dass `v2_opt` deutlich schneller ist als `v1_base`, da **-O3** effizienteren Maschinencode erzeugt.
2. **Modulo**: Wir erwarten, dass `v3_fastmod` die schnellste Variante sein wird. Die Modulo-Operation mit einer Zweierpotenz (16^8) kann durch eine schnelle bitweise UND-Operation (`&`) ersetzt werden, was viel effizienter ist als eine Division.
3. **Datentyp**: Wir erwarten, dass `v4_int128` am langsamsten ist, da unser Prozessor (64-Bit) 128-Bit-Zahlen emulieren muss und der Speicherbedarf für den Cache doppelt so hoch ist.

3. Ergebnisse & Auswertung

Die Messungen wurden mit `hyperfine` durchgeführt. Hier sind die Ergebnisse:

```
Benchmark 1: ./v1_base
Time (mean ± σ):      814.7 ms ± 29.5 ms    [User: 778.3 ms, System: 29.4 ms]
Range (min ... max):  786.9 ms ... 857.3 ms  10 runs

Benchmark 2: ./v2_opt
Time (mean ± σ):      616.6 ms ±  9.5 ms    [User: 580.9 ms, System: 28.1 ms]
Range (min ... max):  599.4 ms ... 629.1 ms  10 runs

Benchmark 3: ./v3_fastmod
Time (mean ± σ):      620.3 ms ± 16.7 ms    [User: 579.8 ms, System: 30.5 ms]
Range (min ... max):  600.1 ms ... 648.4 ms  10 runs

Benchmark 4: ./v4_int128
Time (mean ± σ):      910.0 ms ± 17.6 ms    [User: 849.0 ms, System: 52.5 ms]
Range (min ... max):  884.2 ms ... 944.0 ms  10 runs

Benchmark 5: ./v5_fastmod_no_opt
Time (mean ± σ):      816.1 ms ± 39.5 ms    [User: 775.7 ms, System: 32.3 ms]
Range (min ... max):  793.9 ms ... 913.0 ms  10 runs
```

- **v2_opt (616 ms)** war die schnellste Version (1.32x schneller als Basis).
- **v3_fastmod (620 ms)** war überraschenderweise **nicht schneller** als `v2_opt`.
- **v4_int128 (910 ms)** war die langsamste Version (sogar langsamer als die Basisversion ohne Optimierung).

```
Summary
./v2_opt ran
 1.01 ± 0.03 times faster than ./v3_fastmod
 1.32 ± 0.05 times faster than ./v1_base
 1.32 ± 0.07 times faster than ./v5_fastmod_no_opt
 1.48 ± 0.04 times faster than ./v4_int128
```

Analyse:

1. **Compiler-Optimierung (-O3):** Hat wie erwartet einen großen Einfluss und bringt etwa 30% Leistungssteigerung.
2. **Memory Bound (Speicherlimitierung):** Dass v3_fastmod nicht schneller war, zeigt, dass das Programm nicht durch die Rechenleistung der CPU (ALU), sondern durch den **Speicherzugriff** begrenzt ist. Die CPU wartet auf Daten aus dem RAM, daher bringt eine schnellere Division keinen Vorteil ("Latency Hiding").
3. **int128 Problem:** Die Variante v4_int128 ist langsam, weil sie die **räumliche Lokalität (Spatial Locality)** verschlechtert. Da ein int128 doppelt so groß ist wie int64, passen nur halb so viele Elemente in eine Cache-Line. Das führt zu mehr Cache-Misses und höherem Speicherverkehr.

Aufgabe 1.2: Energie und Kostenvergleich

Um die Effizienz auf unterschiedlicher Hardware zu bewerten, haben wir das Programm zusätzlich auf einem älteren Laptop ausgeführt und die Ergebnisse verglichen.

1. Geräteparameter und Messung

Gerät A (Referenz):

- **Modell:** Apple MacBook (Apple Silicon).
- **Leistungsaufnahme (P_A):** ca. 20 Watt.
- **Gemessene Zeit (t_A):** 0.910 s (Variante v4_int128).

Gerät B (Vergleichsgerät):

- **Modell:** Acer Aspire 3 (A315).
- **Prozessor:** Intel Core i3-6006U (2.0 GHz, Dual Core).
- **Leistungsaufnahme (P_B):** ca. 30 Watt.
- **Gemessene Zeit (t_B):** 2.942 s.

Beobachtung: Das ältere Gerät benötigt für die gleiche Aufgabe mehr als die dreifache Zeit (3.2x).

2. Energieverbrauch ($E = P \cdot t$)

Gerät A (Mac):

$$E_A = 20 \text{ W} \cdot 0.910 \text{ s} \approx 18.2 \text{ Joule}$$

Gerät B (Acer):

$$E_B = 30 \text{ W} \cdot 2.942 \text{ s} \approx 88.3 \text{ Joule}$$

Fazit: Der Acer Aspire 3 verbraucht für die gleiche Rechenoperation fast **5-mal mehr Energie** (88.3 J vs 18.2 J) als das moderne System.

3. Stromkosten

Wir berechnen die Kosten basierend auf dem aktuellen Börsenstrompreis (EPEX SPOT Day-Ahead, DE-LU, ca. 110 €/MWh). Preis pro kWh: 0.11 €.

Kosten Gerät A:

$$K_A = \frac{18.2}{3.6 \cdot 10^6} \text{ kWh} \cdot 0.11 \left(\frac{\text{€}}{\text{kWh}} \right) \approx 5.56 \cdot 10^{-7} \text{ €}$$

Kosten Gerät B:

$$K_B = \frac{88.3}{3.6 \cdot 10^6} \text{ kWh} \cdot 0.11 \left(\frac{\text{€}}{\text{kWh}} \right) \approx 2.70 \cdot 10^{-6} \text{ €}$$

Fazit: Das ältere System ist deutlich ineffizienter und verbraucht fast 5-mal mehr Energie für dieselbe Berechnung.

4. Begründung

Die drastischen Unterschiede lassen sich durch die Speicherhierarchie erklären:

- **CPU Stalls durch Latenz:** Das Programm ist speicherlastig (Memory Bound). Das ältere System hat eine langsamere Anbindung an den Hauptspeicher. Die CPU muss häufiger und länger auf Daten warten ("Stall"), bevor sie weiterrechnen kann. Während dieser Wartezeit wird Energie verbraucht, ohne dass Arbeit verrichtet wird.

- **Räumliche Lokalität:** Die Verwendung von `int128` halbiert die Anzahl der Elemente, die in einen Cache-Block passen. Das erzwingt häufigeres Nachladen aus dem langsamen Hauptspeicher, was das ältere Speichersystem stärker belastet als das moderne.

5. Verfälschende Faktoren

Warum weicht die Theorie von der Praxis ab?

- **Verdrängung im Cache:** Auf dem Laptop laufen nebenher andere Programme (Betriebssystem, Browser etc.). Diese greifen ebenfalls auf den Speicher zu und verdrängen Teile meiner Daten aus dem Cache. Beim nächsten Zugriff müssen diese Daten wieder zeitaufwendig aus dem RAM geholt werden.
- **Cache-Status (Cold vs. Warm):** Die erste Ausführung des Programms ist immer langsamer, da der Cache zunächst leer ("kalt") ist und die Daten erst aus dem Hauptspeicher geladen werden müssen.

Aufgabe 2.1: Block- und Cache-Größe

Gegeben ist ein 8-Bit Adressbus und die Information, dass an jeder Adresse ein **Halfword (2 Bytes)** gespeichert ist. Die Adressaufteilung ist: Tag (3 Bits), Index (3 Bits), Block Offset (2 Bits).

1. Wie groß ist ein Block? Der Block Offset besteht aus 2 Bits. Das bedeutet, ein Block enthält $2^2 = 4$ adressierbare Einheiten ("Wörter"). Da eine Adresse aber auf 2 Bytes verweist, müssen wir dies multiplizieren:

$$\text{Anzahl Wörter pro Block} = 2^2 = 4$$

$$\text{Blockgröße} = 4 \text{ Wörter} \times 2 \text{ Bytes/Wort} = 8 \text{ Bytes}$$

2. Wie groß kann der Cache maximal sein? Der Index besteht aus 3 Bits. Das bestimmt die Anzahl der Cache-Zeilen (Lines), die wir adressieren können.

$$\text{Anzahl Lines} = 2^3 = 8 \text{ Lines}$$

Die Gesamtgröße ist die Anzahl der Lines mal der Größe eines Blocks:

$$\text{Cache-Größe} = 8 \text{ Lines} \times 8 \text{ Bytes/Line} = 64 \text{ Bytes}$$

In KiBiBytes:

$$\frac{64}{1024} = 0.0625 \text{ KiB}$$

Aufgabe 2.2: Veränderte Konfiguration

Hier untersuchen wir die Auswirkungen einer veränderten Blockgröße (1 Word = 4 Bytes) bei gleichbleibender Cache-Gesamtgröße (64 Bytes).

A. Direkt abbildender Cache (Direct Mapped)

Die Blockgröße beträgt nun 4 Bytes. Da eine Adresse weiterhin auf ein Halfword (2 Bytes) verweist, beinhaltet ein Block genau 2 Adressen.

1. Block Offset:

$$\log_2 \left(4 \frac{\text{Bytes}}{2} \text{ Bytes/Adresse} \right) = 1 \text{ Bit}$$

2. **Index:** Die Anzahl der Cache-Lines erhöht sich, da die Blöcke kleiner geworden sind:

$$\text{Anzahl Lines} = 64 \frac{\text{Bytes}}{4} \text{ Bytes/Block} = 16 \text{ Lines}$$

$$\text{Index-Bits} = \log_2(16) = 4 \text{ Bits}$$

3. Tag:

$$8 \text{ Gesamt} - 4 \text{ Index} - 1 \text{ Offset} = 3 \text{ Bits}$$

Neue Aufteilung: Tag: 3 Bits | Index: 4 Bits | Offset: 1 Bit.

B. 2-fach Assoziativer Cache

Bei gleicher Blockgröße (1 Word) werden die 16 Lines nun in Sets zu je 2 Lines ("Ways") organisiert.

1. **Block Offset:** Bleibt gleich (1 Bit).

2. **Index (Set-Index):**

$$\text{Anzahl Sets} = \frac{\text{Anzahl Lines}}{\text{Assoziativität}} = \frac{16}{2} = 8 \text{ Sets}$$

$$\text{Index-Bits} = \log_2(8) = 3 \text{ Bits}$$

3. Tag:

$$8 \text{ Gesamt} - 3 \text{ Index} - 1 \text{ Offset} = 4 \text{ Bits}$$

Neue Aufteilung: Tag: 4 Bits | Index: 3 Bits | Offset: 1 Bit.

C. Adressierbarer Speicher

Frage: Kann man so mehr Speicherblöcke adressieren? **Antwort: Nein.** Die Breite des Adressbusses bleibt unverändert bei 8 Bit. Damit ist der maximal adressierbare Speicherraum physikalisch auf $2^8 = 256$ Adressen (bzw. 512 Bytes) begrenzt. Die interne Organisation des Caches ändert nur, **wie** diese Daten im schnellen Zwischenspeicher abgelegt werden, nicht aber, **wieviele** Hauptspeicher adressiert werden kann.

Aufgabe 2.3

Wir betrachten den ursprünglich gegebenen Cache:

- **Adressbreite:** 8 Bit.
- **Daten:** Halfword (2 Bytes) pro Adresse.
- **Struktur:** Tag (3 Bit) | Index (3 Bit) | Offset (2 Bit).
- **Größe:** 8 Lines, Direct Mapped.

a) Hit oder Miss?

Wir zerlegen jede Adresse binär (Tag - Index - Offset):

- Tag: Bits 7-5
- Index: Bits 4-2
- Offset: Bits 1-0

Adresse	Binär (T-I-O)	Tag	Index	Hit/Miss	Aktion im Cache
0xDC	110 111 00	6	7	Miss	Setze Index 7 auf Tag 6
0x90	100 100 00	4	4	Miss	Setze Index 4 auf Tag 4
0xD3	110 100 11	6	4	Miss	Konflikt! Index 4: Tag 4 → 6
0x0C	000 000 11	0	3	Miss	Setze Index 3 auf Tag 0
0xDD	110 111 01	6	7	Hit	Index 7 enthält Tag 6
0x93	100 100 11	4	4	Miss	Konflikt! Index 4: Tag 6 → 4
0xD0	110 100 00	6	4	Miss	Konflikt! Index 4: Tag 4 → 6
0x0D	000 000 11	0	3	Hit	Index 3 enthält Tag 0

Berechnung der Hit-Rate:

- Anzahl Zugriffe: 8
- Anzahl Hits: 2 (0xDD, 0x0D)

$$\text{Hit-Rate} = \frac{2}{8} = 0.25 = 25\%$$

—

FRAGE: bei D3, 93 zwei Cache Lines wegen Alignment befüllt? Bei D3: D0 D1 D2 D3, Index 05: D4 D5 D6 D7

b) Abschließende Belegung des Caches

Nach der Ausführung aller Befehle sieht der Cache wie folgt aus:

- **Index 3:** Tag 0 (aus 0x0D/0x0C)
- **Index 4:** Tag 6 (aus 0xD0)
- **Index 7:** Tag 6 (aus 0xDC/0xDD)
- **Alle anderen Indizes:** Leer (Invalid)

c) Umwandlung in 2-fach assoziativen Cache

Annahme: Die Adressstruktur (Tag 3 Bit, Index 3 Bit, Offset 2 Bit) bleibt identisch. Das bedeutet, wir haben weiterhin $2^3 = 8$ Sets (Indizes). Da der Cache nun 2-fach assoziativ ist, gibt es pro Set 2 Speicherplätze ("Ways").

1. **Veränderung der Hit-Rate:** Wir analysieren erneut die Konflikte bei Index 4 (0x90, 0xD3, 0x93, 0xD0):

- 0x90 (Tag 4, Set 4): **Miss**. Set 4 speichert [Tag 4, -].
- 0xD3 (Tag 6, Set 4): **Miss**. Set 4 speichert [Tag 4, Tag 6]. (Kein Konflikt, da 2 Plätze!)
- 0x93 (Tag 4, Set 4): **Hit**. Tag 4 ist im Set enthalten.
- 0xD0 (Tag 6, Set 4): **Hit**. Tag 6 ist im Set enthalten.

Die Zugriffe 0x93 und 0xD0, die vorher Misses waren, sind nun Hits.

- Neue Hits: 0xDD, 0x93, 0xD0, 0xD0.
- Neue Anzahl Hits: 4.

$$\text{Neue Hit-Rate} = \frac{4}{8} = 50\%$$

2. **Veränderung der Größe:**

- Anzahl Sets: 8 (bestimmt durch 3 Bit Index).
- Blockgröße: 8 Bytes (bestimmt durch 2 Bit Offset, unverändert).
- Assoziativität: 2 Blöcke pro Set.

$$\text{Neue Größe} = 8 \text{ Sets} \times 2 \text{ Blöcke/Set} \times 8 \text{ Bytes} = 128 \text{ Bytes}$$

Antwort: Die Größe des Caches **verdoppelt sich** von 64 Bytes auf 128 Bytes.

d) Reduzierung von Cache-Misses

Es gibt grundsätzlich zwei Hauptstrategien, um Misses zu reduzieren, die auf unterschiedlichen Prinzipien basieren:

1. **Erhöhung der Blockgröße:**

- **Prinzip: Räumliche Lokalität (Spatial Locality).**
- Wenn auf ein Datum zugegriffen wird, ist es sehr wahrscheinlich, dass kurz darauf auf benachbarte Adressen zugegriffen wird. Größere Blöcke laden diese Nachbarn gleich mit in den Cache ("Prefetching"-Effekt), was Misses bei sequentiellen Zugriffen reduziert.

2. **Erhöhung der Assoziativität:**

- **Prinzip:** Reduzierung von **Conflict Misses**.
- In einem direkt abbildenden Cache können zwei Adressen, die zufällig den gleichen Index haben, sich ständig gegenseitig verdrängen ("Thrashing"), selbst wenn der Cache eigentlich noch leer wäre. Assoziativität erlaubt es, mehrere Blöcke mit gleichem Index gleichzeitig zu speichern.

Aufgabe 3: Cache-Verhalten

```
.data
offset: .word 64
size: .word 8
array: .byte 1, 2, 3, 4, 5, 6, 7, 8

.text
la t0 array    # Beginn des Arrays
lw t1 size     # Größe des Arrays
add t1 t1 t0    # Ende des Arrays
lw t2 offset   # Offset beim Kopieren
add t2 t2 t0    # Beginn des kopierten Arrays

loop:
    bge t0 t1 end
    lb t3 0(t0)
    sb t3 0(t2)
    addi t0 t0 1
    addi t2 t2 1
    j loop
end:
    nop
```

3.1. Mögliche Cache-Konfigurationen (32 Words)

Die Gesamtgröße des Caches beträgt 32 Words. In allen Fällen werden die Strategien Write-back und Write-allocate verwendet.

Konfiguration 1:

- 32 Lines
- 1 Word pro Line
- Direct-mapped

Diese Konfiguration nutzt hauptsächlich temporale Lokalität. Räumliche Lokalität wird kaum ausgenutzt. Konflikt-Misses treten häufig auf.

Konfiguration 2:

- 8 Lines
- 4 Words pro Line
- Direct-mapped

Diese Konfiguration stellt einen guten Kompromiss dar. Sowohl temporale als auch räumliche Lokalität werden genutzt. Die Anzahl der Konflikt-Misses ist moderat.

Konfiguration 3:

- 2 Lines
- 16 Words pro Line
- Direct-mapped

Diese Konfiguration nutzt räumliche Lokalität sehr gut. Allerdings ist die Gefahr von Konflikt-Misses hoch, da nur wenige Cache-Lines vorhanden sind.

—

3.2. Wann ist Write-allocate nicht sinnvoll?

Write-allocate ist nicht sinnvoll, wenn Daten nur geschrieben, aber nicht erneut gelesen werden.

In solchen Fällen führt Write-allocate dazu, dass unnötige Cache-Lines aus dem Hauptspeicher geladen werden.

Typische Beispiele sind:

- Logging
- Streaming-Ausgaben
- Initialisierung großer Speicherbereiche

Hier ist No Write-allocate effizienter.

—

Um Programmieren die Wahl zwischen den beiden Strategien zu überlassen, könnte man die *store* Instruktion in *store with write around* und *store with write allocate* verzweigen:

```
# Syntax ist analog zu bestehenden RISC-V `store` Instruktionen
salw t0 0 t1 # (s)to(re) (w)ord using write (a)llocate
sarw t0 0 t1 # (s)to(re) (w)ord using write (a)r(ou)nd
```

Standard C hat keinen Mechanismus, um eine per-store cache-policy auszudrücken. Denkt man sich entsprechende Befehle aus, so trifft man auf ein Problem:

```
int foo;

// Ein neuer Befehl fuer jede Policy:
// denkbar, aber man kommt in Schwierigkeiten, wenn man die Funktionssignatur
// definieren will.
//
// Versuch mit einem generic <T>type (existiert nicht in C!):
setal((int) &foo, 1); // int setal(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write allocate

setar((int) &foo, 2); // int setar(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write around

foo = 3; // standard syntax: let compiler decide

// Problem: Es laesst sich in C nicht leicht ausdruecken, dass der Typ von `val`
// derselbe wie der des Pointers `dest` ist.
// Wir braechten soetwas wie generic Types.
// Oder eine gesonderte Funktion fuer jeden Datentypen: `setal_int, setal_float,
// setal_char`, usw.
// Die Reibung zeigt, dass diese Modifikation problematisch waere.
```

Es ist auch aus einer weiteren Perspektive fraglich, diesen Mechanismus in dieser Weise in C-Code zu exponieren – es wird hier eine Abstraktionsgrenze überschritten. Angebracht könnte es sein, entsprechende Compiler-Flags zu benutzen. Diese würden dann aber für das ganze translation unit gelten.

Fazit: einem C-Programm die Kontrolle über cache-policy write allocate vs. write around zu geben, ist einfacher gesagt als getan. Global liesse sich das per Compile-Flags erreichen. Aber um dies per store Anweisung zu kontrollieren, wären erhebliche Änderungen nötig (z.B. compiler extensions, oder intrinsics).

3.3. Vergleich der Write-Strategien bei Offset 64 und 128

Der verwendete Cache ist:

- Direct-mapped

- 8 Lines
- 4 Words pro Line

Der verwendete Datencache ist direct-mapped und besteht aus 8 Cache-Lines mit jeweils 4 Words pro Line.

Bei einem Offset von 64 Byte liegen die Quell- und Zieladressen in unterschiedlichen Cache-Lines. Daher treten nur wenige Konflikt-Misses auf.

Bei Verwendung von Write-back und Write-allocate ist die Hit-Rate in diesem Fall gut, da sowohl Lese- als auch Schreibzugriffe von der räumlichen Lokalität profitieren.

Bei Write-through und Write-allocate bleibt die Hit-Rate ebenfalls akzeptabel, jedoch verursacht jeder Schreibzugriff einen zusätzlichen Zugriff auf den Hauptspeicher.

Bei Write-back und No Write-allocate werden Schreibzugriffe nicht im Cache gespeichert. Die Lesezugriffe auf die Quelldaten profitieren weiterhin vom Cache, sodass die Hit-Rate insgesamt mittel ist.

Bei Write-through und No Write-allocate führen alle Schreibzugriffe direkt zu Speicherzugriffen. Dies verschlechtert die Performance, auch wenn keine starken Konflikt-Misses auftreten.

—

Bei einem Offset von 128 Byte entspricht der Abstand zwischen Quell- und Zieladressen der Gesamtgröße des Caches. In einem direct-mapped Cache werden beide Adressen auf dieselbe Cache-Line abgebildet.

Bei Write-back und Write-allocate führt dies dazu, dass sich Quell- und Zieldaten bei nahezu jedem Zugriff gegenseitig aus dem Cache verdrängen. Die Hit-Rate ist daher sehr schlecht.

Bei Write-through und Write-allocate tritt die gleiche Konfliktproblematik auf. Zusätzlich verursachen die Schreibzugriffe einen hohen Speicherverkehr.

Bei Write-back und No Write-allocate werden Zieladressen bei einem Write-Miss nicht in den Cache geladen. Dadurch bleiben die Quelldaten im Cache erhalten, und die Lesezugriffe haben eine gute Hit-Rate.

Auch bei Write-through und No Write-allocate bleiben die Quelldaten im Cache, da Schreibzugriffe den Cache umgehen. In diesem Fall ist die Hit-Rate ebenfalls gut.

Messergebnisse aus der Simulation (Ripes)

Die folgenden Messergebnisse wurden mit dem Simulator Ripes ermittelt. Es wurde ein gleitender Mittelwert über 50 Zyklen verwendet. Die Cache-Konfiguration entspricht der Aufgabenstellung.

—

Offset = 64 Byte

64	Hits	Misses
WB + WA	16	2
WT + WA	16	2
WB + NoWA	9	9
WT + NoWA	9	9

Table 1: Messergebnisse (Offset 64 Byte)

—

Offset = 128 Byte

128	Hits	Misses
WB + WA	2	16
WT + WA	2	16
WB + NoWA	9	9
WT + NoWA	9	9

Table 2: Messergebnisse (Offset 128 Byte)

Die Messergebnisse bestätigen die theoretische Analyse. Bei Offset 128 führen Write-allocate-Strategien zu starken Konflikt-Misses, da Quell- und Zieladressen auf dieselbe Cache-Line abgebildet werden. No Write-allocate verhindert diese Verdrängung und verbessert die Hit-Rate deutlich.

3.4 offset 128 und write allocate

Bei Offset 128 entspricht der Abstand zwischen Quell- und Zieladresse genau der Größe des gesamten Caches.

In einem direct-mapped Cache führt dies dazu, dass beide Adressen auf dieselbe Cache-Line abgebildet werden.

Bei Verwendung von Write-allocate verdrängen sich Quell- und Zieldaten gegenseitig aus dem Cache. Dies führt zu einer sehr niedrigen Hit-Rate.

—

Mathematischer:

Bei $\text{offset} = 128$ werden das `src` und `dest` auf denselben Index gemapped. Für einen Cache mit Assoziativitätsgrad 1 bedeutet das, dass sie auf denselben Index gemapped werden und um dieselbe Line konkurrieren. In der Schleife wird `src` per `load` gecached und dann gleich wieder von `dest` in der `store` Instruktion verdrängt. Das wiederholt sich und wir sehen entsprechende Verschlechterung in der Hit-Rate.

Begründung: Wir benutzen die Formel

$$\text{index} = (\text{address} \gg \text{offset_bits}) \bmod \text{number_of_lines}$$

Für uns ergibt das:

$$\text{index} = (\text{address} \gg 4) \bmod 2^3$$

Index wird also bestimmt durch Bits 4, 5, 6 der Adresse.

Untersuchen wir, was genau bei $\text{address} + 64$ bzw. $\text{address} + 128$, passiert sehen wir, dass wegen $64 = 2^6$; $128 = 2^7$ in dem einen Fall die Relevanten Bits veraendert werden, im anderen allerdings nicht. Resultat: die Speicherbloecke werden einmal verschiedenen Indizes zugeordnet, und einmal demselben.

Man koennte das Problem umgehen, indem man die Assoziativitaet auf 2 anhebt.– Dadurch koennten src und dest im selben Set gecached werden. Moegliche Konfiguration: 4 Sets a 2 Bloecke a 4 Words (Assoziativitaetsgrad 2).

3.5 Cache-Hits maximieren

Die zuendende Idee ist, loads und stores nicht zu verflechten, sondern zu batchen: wir verwenden die ganze geladene Cache Line, erst danach gehen wir zur write-Phase ueber, in der die Line evtl. verdraengt wird. Verdraengung skaliert nun mit $n/\text{size cache line}$ und nicht mehr mit n .

Im Falle unseres Arrays, muessten wir einfach den loop ersetzen:

```
lw t3 0(t0)
lw t4 4(t0)
sw t3 0(t2)
sw t3 4(t2)
```

und kommen damit auf eine Hit-Rate von 67%.

Fuer eine skalierte Version bietet sich eine Schleife an, die mit ganzen Cache-Lines arbeitet:

```
// N = array length in words
// step of 4 -> our cache block size is 4 words
for i in 0..N-1 step 4:
    w0 = load word src[i+0]
    w1 = load word src[i+1]
    w2 = load word src[i+2]
    w3 = load word src[i+3]

    store word dest[i+0] = w0
    store word dest[i+1] = w1
    store word dest[i+2] = w2
    store word dest[i+3] = w3
```

Aufgabe 4: Cache-Kohärenz (Ring-Modell)

Gegeben ist ein 4-Core-System, bei dem Cache-Änderungen über einen Ring weitergegeben werden. Pro passierendem Kern wird ein Takt benötigt, und jeder empfangende Kern leitet die Änderung an anliegende Kerne weiter. Es wird jeweils der „aktuellere“ Wert übernommen.

Erfüllt das Modell sequentielle Kohärenz?

Nein, nicht in allen Fällen.

Gegenbeispiel read own writes:

Ein älteres Datum wird nach store zu Core 0 propagiert:

- t : core 3 sets $x = 3$
- $t+1$: core 0 sets $x = 0$
- $t+2$: core 0 gets propagated value $x = 3$ from 3's write at t
- $t+3$: core 0 reads $x = 3$

Gegenbeispiel write serialization:

- t : core 0 sets $x=0$
- $t+1$: core 2 sets $x=1$; core 0 reads $x=0$
- $t+2$: core 2 reads $x=1$
- $t+3$: $x=0$ propagates to core 2
- $t+4$: core 2 reads $x=0$
- $t+5$: $x=0$ propagates to core 0; core 0 reads $x=0$

Hier 'sieht' Core 2 die Sequenz 1, 0, Core 0 hingegen 0, 1.

Was muss man ändern?

Man benötigt eine **globale Serialisierung** von Writes (total order), damit alle Kerne dieselbe Reihenfolge der Schreiboperationen sehen.

Eine einfache Lösung ist ein **Token**-Mechanismus: Nur der Kern, der aktuell das Token besitzt, darf schreiben. Nach einer Schreiboperation wird das Token weitergereicht. Damit werden alle Writes automatisch in eine eindeutige Reihenfolge gebracht.

Alternative: Jede Schreiboperation bekommt eine globale Sequenznummer (z.B. durch das Token oder einen zentralen Zähler). Updates werden von allen Kernen strikt nach dieser Nummer angewendet (ggf. mit Puffern/Warten), sodass alle Kerne dieselbe Reihenfolge sehen.

Damit wird insbesondere **Write serialization** eingehalten und das Modell kann sequentielle Kohärenz erfüllen.